

Claude Code Cheatsheet

L'essentiel au quotidien pour une productivité maximale

Florian BRUNIAUX

juillet 2026

v1.0.0

Guide v3.42.0



Table des matières

1	Installation	2
2	Commandes essentielles	3
3	Raccourcis clavier	5
4	Références de fichiers	6
5	Fonctionnalités méconnues (mais officielles !)	7
6	Modes de permission	8
7	Mémoire & configuration (2 niveaux)	9
7.1	Structure du dossier .claude/	9
8	Gestion du contexte (CRITIQUE)	10
8.1	Commandes de récupération du contexte	12
9	Plan Mode & réflexion	13
10	Workflow typique	14
11	Formule de prompt rapide	15
12	Serveurs MCP	16
13	Outils de recherche, aide-mémoire	17
14	Aide-mémoire des flags CLI	18
15	Anti-patterns	19
16	Optimisation des coûts	20
17	Arbre de décision rapide	21
17.1	Carte rapide des quatre couches	21
18	Résolution rapide des problèmes courants	23

1 Installation

```
npm install -g @anthropic-ai/claude-code # Vérifier l'installation which claude && claude --version
```

Contrôle santé : `claude doctor && claude mcp list`

2 Commandes essentielles

Commande	Action
<code>/help</code>	Aide contextuelle
<code>/powerup</code>	Leçons animées interactives sur les fonctionnalités de Claude Code
<code>/clear</code>	Réinitialiser la conversation
<code>/compact</code>	Libérer du contexte
<code>/status</code>	État de la session + usage du contexte
<code>/context</code>	Détail de la consommation de tokens
<code>/plan</code>	Entrer en Plan Mode (aucune modification)
Approuver le plan	Sortir du Plan Mode (<code>Shift+Tab</code> pour sortir sans valider)
<code>/model</code>	Changer de modèle (sonnet/opus/opusplan)
<code>/insights</code>	Analytique d'usage + rapport d'optimisation
<code>/simplify</code>	Revue de nettoyage uniquement sur le code modifié : réutilisation, simplification, efficacité
<code>/batch</code>	Refactors à grande échelle via 5 à 30 agents en worktrees parallèles
<code>/teleport</code>	Téléporter une session depuis le web
<code>/tasks</code>	Suivre les tâches en arrière-plan
<code>/remote-env</code>	Configurer un environnement cloud
<code>/rc</code>	Démarrer une session de contrôle à distance (Research Preview, Pro/Max)

Commande	Action
<code>/mobile</code>	Récupérer les liens de téléchargement de l'app mobile Claude
<code>/fast</code>	Basculer le mode rapide (vitesse x2,5, coût x6)
<code>/voice</code>	Activer l'entrée vocale (maintenir Espace pour parler)
<code>/recap</code>	Résumé de contexte de session au retour d'une pause
<code>/effort [niveau]</code>	Profondeur de réflexion : low/medium/high/xhigh/max
<code>/loop [intervalle] [prompt]</code>	Répète un prompt en boucle (10 min par défaut)
<code>/goal [condition]</code>	Mode multi-tours autonome jusqu'à ce que la condition soit remplie
<code>/rename [nom]</code>	Nommer ou renommer la session en cours
<code>/debug</code>	Débogage systématique
<code>/exit</code>	Quitter (ou Ctrl+D)

3 Raccourcis clavier

Raccourci	Action
<code>Shift+Tab</code>	Alternner les modes de permission
<code>Esc × 2</code>	Rewind (annuler)
<code>Ctrl+C</code>	Interrompre
<code>Ctrl+R</code>	Rechercher dans l'historique des commandes
<code>Ctrl+L</code>	Effacer l'écran (garde le contexte)
<code>Ctrl+B</code>	Tâches en arrière-plan
<code>Ctrl+F</code>	Tuer tous les agents en arrière-plan (double appui)
<code>Alt+T</code>	Activer/désactiver la réflexion
<code>Espace</code> (maintenu)	Entrée vocale (nécessite <code>/voice</code>)
<code>Tab</code>	Autocomplétion
<code>Shift+Enter</code>	Nouvelle ligne
<code>Ctrl+D</code>	Quitter

Raccourcis IDE : VS Code `Alt+K` | JetBrains `Cmd+Option+K`

4 Références de fichiers

@chemin/vers/fichier.ts → Référencer un fichier @nom-agent → Appeler un agent
!commande-shell → Exécuter une commande shell

5 Fonctionnalités méconnues (mais officielles !)

Fonctionnalité	Ce que ça fait
Tasks API	Listes de tâches persistantes avec dépendances
Background Agents	Des sous-agents travaillent pendant que vous codez
Agent Teams	Coordination multi-agents (TeamCreate/SendMessage)
Auto-Memories	Capture automatique du contexte entre sessions
Session Forking	Rewind + création d'une timeline parallèle
LSP Tool	Intelligence de code (go-to-def, refs), ~50ms contre 45s avec grep
Mode vocal	Entrée vocale native, transcription gratuite
Remote Control	Contrôler une session locale depuis téléphone/navigateur (Research Preview, Pro/Max)
Tâches planifiées (Cloud/Desktop)	Planification machine éteinte via <code>/schedule</code> ou l'app Desktop
Skill Evals	Tests A/B et calibrage des déclencheurs pour les skills personnalisées
Output Styles	<code>/config</code> → modes Default, Explanatory, Learning
Rules Templates	Templates <code>.claude/rules/</code> prêts à l'emploi (arch/qualité/tests/perf)

Astuce : ce ne sont pas des « secrets », tout est dans le [CHANGELOG](#). Lisez-le !

6 Modes de permission

Mode	Édition	Exécution
Default	Demande	Demande
acceptEdits	Auto	Demande
Plan Mode	Aucune	Aucune
auto	Le classifieur décide	Le classifieur décide
dontAsk	Uniquement si dans les règles autorisées	Uniquement si dans les règles autorisées
bypassPermissions	Auto	Auto (CI/CD uniquement)

Shift+Tab pour changer de mode

7 Mémoire & configuration (2 niveaux)

Niveau	Chemin	Portée	Git
Projet	<code>.claude/</code>	Équipe	Oui
Personnel	<code>~/.claude/</code>	Vous (tous projets)	Non

Priorité : le projet prime sur le personnel

7.1 Structure du dossier `.claude/`

`.claude/`

├─ CLAUDE.md

Mémoire locale (gitignored)

├─ settings.json

Hooks (versionné)

├─ settings.local.json

Permissions (non versionné)

├─ agents/

Agents personnalisés

├─ hooks/

Scripts d'événements

├─ rules/

Règles chargées automatiquement

| └─ architecture-review.md

| └─ code-quality-review.md

| └─ test-review.md

| └─ performance-review.md

└─ skills/

Slash commands + modules de connaissance (unifiés)

8 Gestion du contexte (CRITIQUE)

Statusline : Model: Sonnet | Ctx: 89.5k | Ctx(u): 56.0%

✅ 0-50%	⚠️ 50-70%	💎 70-90%	🔴 90%+

Surveillez `Ctx(u):` → >70% = `/compact` , >85% = `/clear`

Signe	Action
Réponses courtes	<code>/compact</code>
Oublis fréquents	<code>/clear</code>
Contexte >70%	<code>/compact</code>
Tâche terminée	<code>/clear</code>

8.1 Commandes de récupération du contexte

Commande	Usage
<code>/compact</code>	Résumer et libérer du contexte
<code>/clear</code>	Repartir de zéro
<code>/rewind</code>	Annuler les changements récents
<code>claude -c</code>	Reprendre la dernière session
<code>claude -r <id></code>	Reprendre une session spécifique

9 Plan Mode & réflexion

Fonctionnalité	Activation	Usage
Plan Mode	<code>Shift+Tab × 2</code> ou <code>/plan</code>	Explorer sans modifier
OpusPlan	<code>/model opusplan</code>	Opus pour planifier, Sonnet pour exécuter

Opus 4.8 : l'effort par défaut est **xhigh**. Utilisez `ultrathink` pour forcer l'effort maximum sur le prochain tour.

Contrôle	Action	Persistance
Alt+T	Activer/désactiver la réflexion	Session
/config	Activer/désactiver globalement	Permanent
Curseur <code>/effort</code>	low/medium/high/xhigh/max	Session
Paramètre <code>effort</code>	API uniquement, par requête	Par requête

Astuce coût : pour les tâches simples, Alt+T pour désactiver la réflexion, plus rapide et moins cher.

10 Workflow typique

1. Démarrer la session → `claude`
2. Vérifier le contexte → `/status`
3. Plan Mode → `Shift+Tab` × 2 (tâches complexes)
4. Décrire la tâche → prompt clair et précis
5. Vérifier les diffs → toujours lire avant d'accepter !
6. Accepter/Rejeter → `y/n`
7. Vérifier → lancer les tests
8. Commit → une fois la tâche terminée
9. `/compact` → quand le contexte dépasse 70%

11 Formule de prompt rapide

QUOI : [Livrable concret] OÙ : [Chemins de fichiers] COMMENT : [Contraintes, approche]
VÉRIFIER : [Critères de succès]

Exemple :

Ajouter une validation d'entrée au formulaire de connexion. OÙ : src/components/LoginForm.tsx
COMMENT : utiliser un schéma Zod, afficher les erreurs inline
VÉRIFIER : email vide affiche une erreur, format invalide affiche une erreur

12 Serveurs MCP

Serveur	Usage
Serena	Indexation + mémoire de session + recherche de symboles
grepai	Recherche sémantique + analyse du graphe d'appels
Context7	Documentation des librairies
Sequential	Raisonnement structuré
Playwright	Automatisation navigateur
Postgres	Requêtes base de données
doobidoo	Mémoire sémantique + Knowledge Graph

Mémoire Serena : `write_memory()` / `read_memory()` / `list_memories()`

Vérifier l'état : `/mcp`

13 Outils de recherche, aide-mémoire

Décision rapide : texte exact → `rg` | nom exact → `rg` /Serena | concept → `grepai` | structure → `ast-grep`

Tâche	Outil
Trouver du texte exact	<code>rg "TODO"</code>
Trouver du code lié à l'auth	<code>grepai search "authentication"</code>
Qui appelle une fonction	<code>grepai trace callers "login"</code>
Structure d'un fichier	<code>serena get_symbols_overview</code>

14 Aide-mémoire des flags CLI

Flag	Usage
<code>-p "requête"</code>	Mode non-interactif (CI/CD)
<code>-c</code> / <code>--continue</code>	Continuer la dernière session
<code>-r</code> / <code>--resume <id></code>	Reprendre une session spécifique
<code>--teleport</code>	Téléporter une session depuis le web
<code>--model sonnet</code>	Changer de modèle
<code>--add-dir ../lib</code>	Autoriser l'accès hors du répertoire courant
<code>--permission-mode plan</code>	Plan mode
<code>--worktree</code> / <code>-w</code>	Lancer dans un git worktree isolé
<code>--max-budget-usd 5.00</code>	Plafond de dépense API (mode print)
<code>--dangerously-skip-permissions</code>	Auto-accepter (à manier avec précaution)
<code>--mcp-debug</code>	Déboguer les serveurs MCP
<code>--allowedTools "Edit,Read"</code>	Liste blanche d'outils
<code>--debug</code>	Sortie de débogage

15 Anti-patterns

❌ Don't

- Prompts vagues
- Accepter sans lire
- Ignorer les avertissements
- Sauter les permissions
- Contraintes négatives uniquement

✅ Do

- Préciser fichier + ligne avec @références
- Lire chaque diff
- Utiliser /compact à 70%
- Jamais en production
- Fournir des alternatives

16 Optimisation des coûts

Modèle	Usage	Coût
Haiku	Corrections simples, revues	\$
Sonnet	La majorité du développement	\$\$
Opus	Architecture, bugs complexes	\$\$\$
OpusPlan	Plan (Opus) + Exécution (Sonnet)	\$\$

17 Arbre de décision rapide

17.1 Carte rapide des quatre couches

Couche	Question de choix	Référence
Modèle	Quel raisonnement et quel coût conviennent?	Glossaire
Runtime harness	Qui exécute la boucle outils-observation-décision?	Agent Harness Map
Repository harness	Quelles instructions, vérifications et états encadrent le dépôt?	Agent Harness Engineering
Orchestrateur	Faut-il coordonner plusieurs runtimes ou sessions?	Agent Tools

Choisir d'abord le runtime pour une tâche de code. Ajouter un orchestrateur lorsque la coordination est le besoin identifié, pas parce que le catalogue est large.

Tâche simple → demander directement à Claude Tâche complexe → passer par Tasks API pour planifier
Changement risqué → Plan Mode d'abord Tâche répétitive → créer un agent ou une commande
Contexte plein → /compact ou /clear Besoin de doc → utiliser Context7 MCP
Analyse profonde → utiliser Opus (réflexion activée par défaut)

The Golden Rules

- 1 Toujours relire les diffs avant d'accepter
- 2 Utiliser /compact avant que le contexte devienne critique (>70%)
- 3 Être précis dans les demandes (QUOI, OÙ, COMMENT, VÉRIFIER)
- 4 Plan Mode d'abord pour les tâches complexes ou risquées

- 5 Créer un CLAUDE.md pour chaque projet
- 6 Commit fréquemment après chaque tâche terminée
- 7 Savoir ce qui est envoyé : prompts, fichiers, résultats MCP vers Anthropic

18 Résolution rapide des problèmes courants

Problème	Solution
« Command not found »	Vérifier le PATH, réinstaller npm global
Contexte trop élevé (>70%)	<code>/compact</code> immédiatement
Réponses lentes	<code>/compact</code> ou <code>/clear</code>
MCP ne fonctionne pas	<code>claude mcp list</code> , vérifier la config
Permission refusée	Vérifier <code>settings.local.json</code>
Hook bloquant	Vérifier le code de sortie du hook, revoir la logique

Contrôle santé : `which claude && claude doctor && claude mcp list`

Auteur : Florian BRUNIAUX | [Méthode Aristote](#) | Écrit avec Claude
Version 3.42.0 | Août 2026